SplitGP: Achieving Both Generalization and Personalization in Federated Learning

Dong-Jun Han*
Purdue Univirsity
djhan930@gmail.com

Do-Yeon Kim
KAIST
dy.kim@kaist.ac.kr

Minseok Choi
Kyung Hee University
choims@khu.ac.kr

Christopher G. Brinton
Purdue University
cgb@purdue.edu

Jaekyun Moon
KAIST
jmoon@kaist.edu

IEEE INFOCOM 2023 - IEEE Conference on Computer Communications | 979-8-3503-3414-2/23/$31.00 ©2023 IEEE | DOI: 10.1109/INFOCOM53939.2023.10229027

***Abstract*—A fundamental challenge to providing edge-AI services is the need for a machine learning (ML) model that achieves personalization (i.e., to individual clients) and generalization (i.e., to unseen data) properties concurrently. Existing techniques in federated learning (FL) have encountered a steep tradeoff between these objectives and impose large computational requirements on edge devices during training and inference. In this paper, we propose SplitGP, a new split learning solution that can simultaneously capture generalization and personalization capabilities for efficient inference across resource-constrained clients (e.g., mobile/IoT devices). Our key idea is to split the full ML model into client-side and server-side components, and impose different roles to them: the client-side model is trained to have strong personalization capability optimized to each client's main task, while the server-side model is trained to have strong generalization capability for handling all clients' out-of-distribution tasks. We analytically characterize the convergence behavior of SplitGP, revealing that all client models approach stationary points asymptotically. Further, we analyze the inference time in SplitGP and provide bounds for determining model split ratios. Experimental results show that SplitGP outperforms existing baselines by wide margins in inference time and test accuracy for varying amounts of out-of-distribution samples.**

I. Introduction

With the increasing prevalence of mobile and Internet-of Things (IoT) devices, there is an explosion in demand for machine learning (ML) functionality across the intelligent network edge. From the service provider's perspective, providing a high-quality edge-AI service to individual clientsis of paramount importance: given newly collected data, the goal of each client is to apply the provided ML model for inference/decisioning. However, there are two critical challenges that need to be handled to satisfy the client needs in practical edge-AI settings.

Issue 1: Personalization vs. generalization. First, during the inference stage (i.e., after training has completed), each client should be able to make reliable predictions not only for dominant data classes which have been observed locally, but also occasionally for the classes that have not previously appeared in its local data. We refer to these as a client's main classes and out-of-distribution classes, respectively. Federated learning (FL) [1]–[3], the most recently popularized technique for distributing ML across edge devices, has demonstrated a sharp tradeoff between these objectives. In particular, existing works have aimed to create either a *generalized global model* [3]–[17] that is tuned to the data distribution across all clients, or *personalized local models* [18]–[22] that work case-by-case on each client's individual data. For example, a global activity recognition classifier for wearables learned via FL would be optimized for classes of activities observed over all users. We term the capability to classify all classes as "generalization". The generalized global model is a good option when the input data distribution appearing at each client during inference resembles the global training distribution. However, when the data distributions across clients are significantly non-IID (independent and identically distributed), the globally aggregated FL model may not be the best option for many clients (e.g., consider activity sensors for individuals playing different types of sports). Personalized FL approaches tackle this problem by providing a customized local model to each client based on their individual local data distributions (e.g., a basketball vs. football player). We term this capability to classify the local classes as "personalization".

However, when a client needs to make predictions for classes that are not in its local data (i.e., due to distribution shift), the personalized FL model shows much lower performance than the generalized model (see Sec. VI). Hence, it is important to capture both personalization (for handling local classes) and generalization (for handling out-of-distribution classes) in practice where not only the main classes but also the out-of-distribution classes appear occasionally during inference.

Issue 2: Inference requirements. Mobile edge and IoT devices suffer from limited storage and computation resources. As a result, it is challenging to deploy large-scale models (e.g., neural networks with millions of parameters) at individual clients for inference tasks without incurring significant costs. Deploying the full model at a nearby edge server can be another option, but this approach requires direct transmissions of raw data from the client during inference, and can also incur noticeable latency. Moreover, under this framework, when client models are personalized, the server would need to store all of these variations, which presents scalability challenges.

These two issues are thus significant obstacles to high quality edge-AI services, with existing approaches falling short of addressing them simultaneously. Motivated by this, we pose the following research question: *How can we achieve both learning personalization and generalization across resource-constrained edge devices for high-quality inference?*

Overview of approach. To address this question, we propose SplitGP, a new split learning (SL) solution for

* Work performed when Dong-Jun was a postdoctoral researcher at KAIST.

Authorized licensed use limited to: Hongik Univ. Downloaded on May 02,2025 at 07:59:09 UTC from IEEE Xplore. Restrictions apply.

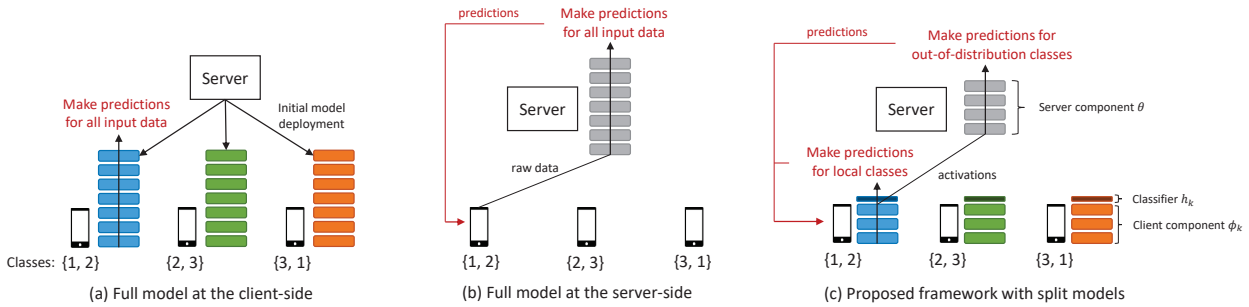


Fig. 1: **Comparison of inference procedures:** Deploying the full model at individual clients (as in Fig. 1(a)) is challenging in resource-constrained edge-AI settings as it induces significant storage and computational burden during inference. When the model is implemented at the server (as in Fig. 1(b)), raw data should be directly transmitted from the client to the server during inference, which incurs various privacy, communication and latency issues. The proposed framework based on model splitting and edge computing in Fig. 1(c) captures both learning personalization and generalization requirements in resource-constrained scenarios while retaining desirable privacy and latency properties.

generalization and personalization in FL settings. Our key idea is to split the full model into two parts, client-side and server-side, and impose different roles to them during inference. The client-side model should have strong *personalization capability*, where the goal is to work well on each user's local distribution. On the other hand, the server-side model, shared by all clients in the system, should have strong *generalization capability* across the tasks of all users. During inference, each client solves its personalized task, i.e., for its main classes, using the client-side model. When the client has to make a prediction that is not related to its personalized task, i.e., for out-of-distribution classes, they can send the output feature from the client-side model to the server-side model, and receive the predicted result back from the edge server. We will show that this combination of model splitting and edge computing captures both personalization and generalization while reducing the storage, computational load and latency during inference compared to existing methods where the full model is provided to individual clients. SplitGP also has significant advantages in terms of privacy and latency compared to schemes where the full model is deployed at the server-side. Fig. 1 compares the inference stage of SplitGP with these existing frameworks; note that the models in Figs. 1(a)&(b) are realized through existing FL and SL methodologies.

Summary of contributions. To the best of our knowledge, simultaneous training of client/server-side models with different roles (personalization and generalization) has not been considered before. Existing works in distributed ML also tend to focus on the training process, without considering the inference stage of clients with small storage space and small computing powers. Overall, our contributions are summarized as follows:

- We propose SplitGP (Sec. III), a hybrid federated and split learning solution that captures both learning generalization and personalization needs with multi-exit neural networks for inference at resource-constrained clients.
- We analytically characterize the convergence behavior of SplitGP (Sec. IV), showing that training at each client will converge to a stationary point asymptotically under common assumptions in distributed ML.
- We conduct a latency analysis of SplitGP (Sec. V), which leads to guidelines on model splitting and insights on the power-rate regime where our scheme is beneficial.

Experimental results (Sec. VI) show that SplitGP outperforms existing approaches in practical scenarios with wide margins of improvement in testing accuracy and inference time.

II. Related Works

Federated learning. A large number of FL techniques [3]–[10], [14]–[17] including FedAvg [3], FedProx [4], FedMA [5], FedDyn [6] and SCAFFOLD [7] have been proposed to construct generalized global models. Personalized FL [18]–[22] has been studied more recently through techniques such as multi-task learning [18], interpolation and finetuning [19], meta-learning [20], and regularization [22]. However, these strategies do not achieve generalization and personalization simultaneously, and thus are not the best options in practice where not only the main classes and but also the out-of-distribution classes appear occasionally during inference. In this respect, a recent work [23] proposed simultaneously constructing generalized and personalized models in FL with two models sharing the same feature extractor. In practice, the full model would need to be deployed at each client for achieving personalization during inference, which is challenging in the edge-AI scenarios we consider with resource-constrained clients.

Split learning. Recently, various SL schemes have been proposed [24]–[29] to reduce client-side storage and computation requirements during training compared to FL. The training process of our approach draws from concepts in SL in that we divide the full model into client-side and server-side components. However, existing works on SL including [26]–[28] do not focus on capturing both generalization and personalization simultaneously. Compared to existing works, we consider a new inference scenario where the clients should make predictions frequently for the main classes but also occasionally for the out-of-distribution classes, and design a solution tailored to this setup. We also let clients to frequently make predictions using only the client-side model (instead of the full model), which results in reduced inference time.

Efficient edge-AI inference. Only a few prior works in distributed ML have focused on the inference stage of the clients at the edge. [30] proposed to deploy distributed deep neural networks on the server and devices during inference, using multi-exit neural networks [31]–[35]. Our approach also borrows the concept of multi-exit neural networks with two

Authorized licensed use limited to: Hongik Univ. Downloaded on May 02,2025 at 07:59:09 UTC from IEEE Xplore. Restrictions apply.

exits to make predictions both at the client-side and at the server-side. However, these previous works have assumed that the model training phase occurs in a centralized manner, which does not consider the important challenge of non-IID local datasets in FL/SL setups where raw data remains at the devices. In practical settings where each client observes main and out-of-distribution classes during inference, incorporating personalization and generalization results in significant performance enhancements during inference, as we will see in Sec. VI.

III. Proposed SplitGP Algorithm

Let K be the number of clients in the system and D_k be the local dataset of client $k = 1, 2, \ldots, K$ to be used for training an ML model. We denote the *full model* as a parameter vector w, which is split into *client-side* and *server-side* model components ϕ_k and θ, respectively. Each client also maintains an *auxiliary classifier* h_k, with output dimension equal to the number of classes, which enables each client k to make predictions using only ϕ_k and h_k; as shown in Fig. 1(c), the output of ϕ_k becomes the input of h_k, and prediction can be made at the output of h_k. As in FL, model training will proceed in a series of training rounds, which we index $t = 0, 1..., T-1$.

Before training begins, we split the initialized full model w^0 into $w^0 = [\phi^0, \theta^0]$, and also initialize h^0. Each client k receives ϕ^0, h^0 and sets $\phi_k^0 = \phi^0$, $h_k^0 = h^0$, whereas θ^0 is deployed at the server. After T global rounds of training, each client k obtains ϕ_k^T, h_k^T, while the server obtains θ^T. We let

$$v_k^t = [\phi_k^t, h_k^t, \theta^t] \tag{1}$$

be the model components obtained at client k and the server when global round t is finished.

Inference scenario and goal. We consider a scenario having distribution shift between training and inference in each client: each client should make predictions mainly for the local classes but also occasionally for the out-of-distribution classes due to distribution shift. We introduce a parameter for the relative portion of out-of-distribution test samples, which is defined as

$$\rho = \frac{\text{\# of out-of-distribution test samples}}{\text{\# of main test samples}}. \tag{2}$$

Compared to the previous works on personalized FL focusing on $\rho = 0$, we consider a practical setup with $\rho > 0$ caused by distribution shift between training and inference.

As depicted in Fig. 1(c), the goal of the k-th client's model, ϕ_k combined with h_k, is to make a reliable prediction for local classes in D_k. The goal of each full model, ϕ_k combined with θ, is to make make a reliable prediction for all classes in the network-wide dataset, $D = \cup_{k=1}^K D_k$, to handle the out-of-distribution classes of each client.

A. Multi-Exit Objective Function

Based on the three model components $v = [\phi, h, \theta]$, we first define the following two losses computed based on D_k.

Client-side loss. Given k-th client's local data D_k and $v = [\phi, h, \theta]$, the client-side loss $\ell_{C,k}(v)$ is defined as

$$\ell_{C,k}(v) = \frac{1}{|D_k|} \sum_{x \in D_k} \ell(x; \phi, h), \tag{3}$$

where $\ell(x; \phi, h)$ is the loss (e.g., cross-entropy loss) computed with the client model (ϕ combined with h) using input data x. (3) is computed by client k.

Server-side loss. We also define the server-side loss $\ell_{S,k}(v)$ computed with the k-th client's local data D_k, as follows:

$$\ell_{S,k}(v) = \frac{1}{|D_k|} \sum_{x \in D_k} \ell(x; \phi, \theta). \tag{4}$$

Here, $\ell(x; \phi, \theta)$ is the loss computed at the output of the full model (ϕ combined with θ) based on input x. As in existing SL schemes, (4) is computed by the server in SplitGP. To facilitate this, for each $x \in D_k$, the client transmits the output features it computes from ϕ_k along with the label to the server[1].

Proposed objective function. In this way, the model ϕ maintained at the client-side affects both the client-side loss $\ell_{C,k}(v)$ and the server-side loss $\ell_{S,k}(v)$. By viewing the model $v = [\phi, h, \theta]$ as a multi-exit neural network [31]–[35] with two exits ($\ell_{C,k}$ and $\ell_{S,k}$), we update ϕ, h, θ to minimize the weighted sum of client/server-side losses computed with D_k:

$$F_k(v) = \gamma \ell_{C,k}(v) + (1-\gamma)\ell_{S,k}(v). \tag{5}$$

Here, γ and $1-\gamma$ correspond to the weights of the client-side loss and the server-side loss, respectively. If $\gamma = 0$, the client-side model is updated only considering the server-side loss, which corresponds to the objective function of SplitFed proposed in [26]. Personalization capability is not guaranteed at the client-side in this case. If $\gamma = 1$, the client-side model does not consider the server-side loss at all, which does not guarantee generalization capability at the server-side. In multi-exit network literatures [33]–[35], a common choice is to give equal weights[2] to both exits with $\gamma = 0.5$.

B. Personalization and Generalization Training

Model update. In the beginning of global round t, we have $v_k^t = [\phi_k^t, h_k^t, \theta^t]$, where ϕ_k^t and h_k^t are implemented at client k while θ^t is deployed at the server. Based on the proposed objective function (5), the models of client k (ϕ_k^t and h_k^t) and the shared server-side model θ^t are updated according to

$$\phi_k^{t+1} = \phi_k^t - \eta_t \tilde{\nabla}_\phi F_k(v_k^t), \tag{6}$$

$$h_k^{t+1} = h_k^t - \eta_t \tilde{\nabla}_h F_k(v_k^t), \tag{7}$$

$$\theta_k^{t+1} = \theta^t - \eta_t \tilde{\nabla}_\theta F_k(v_k^t), \tag{8}$$

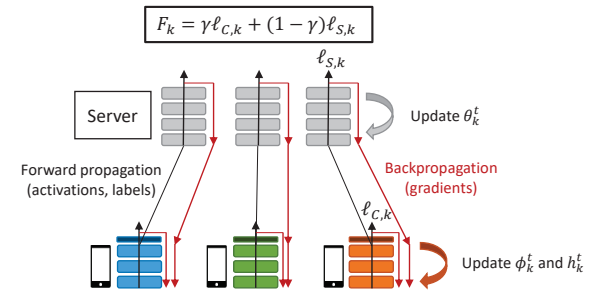
where η_t is the learning rate at global round t, and $\tilde{\nabla} F_k(v_k^t) = \frac{1}{|\tilde{D}_k^t|} \sum_{x \in \tilde{D}_k^t} (\gamma \nabla \ell_C(v; x) + (1-\gamma)\nabla \ell_S(v; x))$ is the stochastic gradient computed with a specific mini-batch $\tilde{D}_k^t \subset D_k$. Fig. 2(a) shows the model update process at client k.

Server-side model aggregation. The updated server-side models based on (8) are aggregated according to $\theta^{t+1} \leftarrow \sum_{i=1}^K \alpha_i \theta_i^{t+1}$, to construct a single server-side model, where $\alpha_i = \frac{|D_i|}{\sum_{k=1}^K |D_k|}$ is the relative dataset size. This is a natural choice to capture generalization capability at the server using a single model.

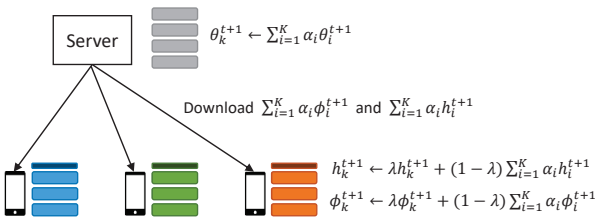
[1]Potential privacy issues can be handled by adding a noise layer [36] at the client as in [26], which constructs private/noisy versions of output features.

[2]Our work can be combined with existing strategies that consider different weights for each exit's loss [32] to further improve the performance.

Authorized licensed use limited to: Hongik Univ. Downloaded on May 02,2025 at 07:59:09 UTC from IEEE Xplore. Restrictions apply.



(a) Loss computation and parameter update process. The overall model can be viewed as a multi-exit neural network with two exits.



(b) Model aggregation process.

Fig. 2: Training process of SplitGP at global round t. After updating the models using $F_k(v_k^t)$ as in Fig. 2(a), the server-side models and the client-side models/classifiers are aggregated according to Fig. 2(b).

Client-specific model aggregation. For client k, ϕ_k combined with h_k should work well on its local classes (personalized task), while ϕ_k combined with θ should work well on all classes in the system. While the updated ϕ_k using (5) enables the client-side model (ϕ_k combined with h_k) to have strong personalization capability, it does not guarantee the generalization performance of the full model (ϕ_k combined with θ_k). In particular, since ϕ_k is updated only with the local data D_k of client k, the output of ϕ_k (which becomes the input of θ in the full model) does not provide meaningful output features for the classes outside of D_k.

A natural way to resolve this issue would be to aggregate ϕ_k^{t+1} for all k as $\sum_{i=1}^K \alpha_i \phi_i^{t+1}$, and deploy this aggregated model at each client. However, this can reduce the personalization capability at each client. In order to capture personalization while providing a meaningful result to the server-side model θ, in SplitGP, each client k computes weighted sum of ϕ_k^{t+1} and the average of ϕ_k^{t+1} for all $k = 1, 2, \ldots, K$, as follows:

$$\phi_k^{t+1} \leftarrow \lambda \phi_k^{t+1} + (1-\lambda) \sum_{i=1}^K \alpha_i \phi_i^{t+1}. \tag{9}$$

Here, $\lambda \in [0, 1]$ controls the weights for personalization and generalization. If $\lambda = 1$, the client-side model has a strong personalization capability but does not provide a meaningful output feature to the server-side model. If $\lambda = 0$, the client-side model provides a generalizable feature to the server but lacks personalization capability. Using λ, the auxiliary classifiers $\{h_k\}_{k=1}^K$ are also aggregated at each client k according to

$$h_k^{t+1} \leftarrow \lambda h_k^{t+1} + (1-\lambda) \sum_{i=1}^K \alpha_i h_i^{t+1}, \tag{10}$$

which enables the client-side model to make reliable predictions on the out-of-distribution classes. Although generalization is not the main goal of the client model, conducting inference for out-of-distribution classes at the client when possible will further reduce communication cost and latency. Moreover, during inference, the client does not automatically know whether a datapoint is from one of its main classes or not. We therefore introduce a confidence threshold for the inference stage in Sec. III-C which chooses between client and server-side inference.

Algorithm 1 SplitGP: Training and Inference

Training Phase

Input: Initialized models $v^0 = [\phi^0, h^0, \theta^0]$
Output: $v_k^T = [\phi_k^T, h_k^T, \theta^T]$ for each client $k = 1, 2, \ldots, K$
for each global round $t = 0, 1, \ldots, T-1$ **do**
  for $k \in \{1, 2, \ldots, K\}$ **in parallel do**
    $\ell_{C,k}(v_k^t) = \frac{1}{|\tilde{D}_k^t|} \sum_{x \in \tilde{D}_k^t} \ell(x; \phi_k^t, h_k^t)$ // Client-side loss
    $\ell_{S,k}(v_k^t) = \frac{1}{|\tilde{D}_k^t|} \sum_{x \in \tilde{D}_k^t} \ell(x; \phi_k^r, \theta_k^t)$ // Server-side loss
    $F_k(v_k^t) = \gamma \ell_{C,k}(v_k^t) + (1-\gamma)\ell_{S,k}(v_k^t)$ // Multi-exit loss
    $\phi_k^{t+1} = \phi_k^t - \eta_r \tilde{\nabla}_\phi F_k(v_k^t)$, $h_k^{t+1} = h_k^t - \eta_r \tilde{\nabla} F_k(v_k^t)$, $\theta_k^{t+1} = \theta_k^t - \eta_r \tilde{\nabla}_\theta F_k(v_k^t)$ // Model update
  end for
  $\theta^{t+1} \leftarrow \sum_{i=1}^K \alpha_i \theta_i^{t+1}$ // Server model aggregation
  $\phi_k^{t+1} \leftarrow \lambda \phi_k^{t+1} + (1-\lambda) \sum_{i=1}^K \alpha_i \phi_i^{t+1}$
  $h_k^{t+1} \leftarrow \lambda \phi_k^{t+1} + (1-\lambda) \sum_{i=1}^K \alpha_i h_i^{t+1}$ // Client model aggregations; λ controls the weights for personalization/generalization
end for
$v_k^T = [\phi_k^T, h_k^T, \theta^T]$

Inference Phase

Input: Test sample z at client k with $v_k^T = [\phi_k^T, h_k^T, \theta^T]$
Output: Prediction result for test sample z
$E_k(z) = -\sum_{q=1}^Q p_k^{(q)}(z) \log p_k^{(q)}(z)$
if $E_k(z) < E_{th}$ **then**
  Make prediction with ϕ_k^T combined with h_k^T
else
  Make prediction with ϕ_k^T combined with θ^T
end if

Fig. 2(b) summarizes the model aggregation step of our scheme. Note that, for simplicity of presentation, we have presented the model updates in (6), (7), (8) assuming a single gradient step at each time t. In practice, these can be repeated multiple times in-between each model aggregation process.

After repeating the overall process for T global rounds, we obtain K different personalized models $\{\phi_k^T\}_{k=1}^K$ and classifiers $\{h_k^T\}_{k=1}^K$, and one server model θ^T. ϕ_k^T and h_k^T are deployed at client k while θ^T is implemented at the edge server.

C. Client-Side and Server-Side Inference

During inference, each client k must determine whether to rely on the client-side (ϕ_k^T, h_k^T) or server-side (ϕ_k^T, θ^T) model. Given a test sample z at client k, the Shannon entropy is first computed using the client-side model (ϕ_k^T, h_k^T) as $E_k(z) = -\sum_{q=1}^Q p_k^{(q)}(z) \log p_k^{(q)}(z)$, where Q is the total number of classes in the system and $p_k^{(q)}(z)$ is the softmax output for class q on sample z, using the model deployed at client k. If

$$E_k(z) \leq E_{th} \tag{11}$$

holds for a desired entropy threshold E_{th}, the inference is made at the client-side. Otherwise, i.e., if $E_k(z) > E_{th}$, the output feature of ϕ_k^T computed on sample z is sent to the server and

Authorized licensed use limited to: Hongik Univ. Downloaded on May 02,2025 at 07:59:09 UTC from IEEE Xplore. Restrictions apply.

the output of the server model θ^T is used for inference. The value of E_{th} in (11) is therefore a control parameter for the amount of communication over the network during inference, while λ in (9) controls the weights for personalization and generalization. We will analyze the effects of E_{th} and λ on SplitGP in Sec. VI. The overall training process and inference stage of our scheme is described in Algorithm 1.

IV. Convergence Analysis

We analyze the convergence behavior of SplitGP based on some standard assumptions in FL [37]–[39].

Assumption 1. *For each k, $F_k(v)$ is L-smooth, i.e., $\|\nabla F_k(u) - \nabla F_k(v)\| \leq L\|u - v\|$ for any u and v.*

Assumption 2. *For each k, the expected squared norm of stochastic gradient is bounded, i.e., $\mathbb{E}[\|\tilde{\nabla} F_k(v)\|^2] \leq G$.*

Assumption 3. *The variance of the stochastic gradient of D_k is bounded, i.e., $\mathbb{E}[\|\nabla F_k(v) - \tilde{\nabla} F_k(v)\|^2] \leq \sigma_k^2$.*

We also define the global loss function $F(v)$ as

$$F(v) = \frac{1}{K}\sum_{k=1}^{K} F_k(v), \tag{12}$$

which is the average of the losses defined in (5). We show that our algorithm converges to a stationary point of (12), which guarantees the generalization capability of SplitGP while including personalization through λ for any non-convex ML loss function $F(v)$.

A. Main Theorem and Discussions

The following theorem gives the convergence behavior of SplitGP. The proof is given in Sec. IV-B.

Theorem 1. ***(SplitGP Convergence)*** *Let $\eta_t = \frac{\eta_0}{a+t}$, where $a = \frac{c+4}{1-\lambda^2}$ for some constant $c > 0$. Suppose that η_0 is chosen to satisfy $\eta_t \leq \frac{1}{2L}$. SplitGP model training converges as*

$$\frac{1}{\Gamma_T}\sum_{t=0}^{T-1}\sum_{k=1}^{K}\frac{\eta_t}{4K}\mathbb{E}\Big[\|\nabla F(v_k^t)\|^2\Big] \leq \frac{F(v^0) - F^*}{\Gamma_T} + \frac{L\sum_{k=1}^{K}\sigma_k^2}{K}\left(\frac{1}{\Gamma_T}\sum_{t=0}^{T-1}\eta_t^2\right) + \epsilon(\lambda)\left(\frac{1}{\Gamma_T}\sum_{t=0}^{T-1}\eta_t^3\right), \tag{13}$$

where

$$\epsilon(\lambda) = \frac{16(c+4)G^2L^2\lambda^2(2-\lambda^2)}{c(1-\lambda^2)^2}, \tag{14}$$

$\Gamma_T = \sum_{t=0}^{T-1}\eta_t$ and F^* is the minimum value of $F(v)$ in (12).

Here, $\epsilon(\lambda)$ is the term specific to our work, arising from the joint consideration of generalization and personalization. By setting $\eta_t = \frac{\eta_0}{a+t}$, we have $\Gamma_T = \sum_{t=0}^{T-1}\eta_t \to \infty$ as T grows, and $\sum_{t=0}^{\infty}\eta_t^2 < \infty$, $\sum_{t=0}^{\infty}\eta_t^3 < \infty$. Hence, for any $\lambda \in [0, 1)$, the upper bound in (13) goes to 0 as T grows. Thus, we have $\min_{t\in\{0,1,\ldots,T-1\}} \mathbb{E}[\|\nabla F(v_k^t)\|] \overset{T\to\infty}{\longrightarrow} 0$ for all $k = 1, ..., K$, which guarantees convergence to a stationary point of (12).

Theorem 1 indicates that $v_k^t = [\phi_k^t, h_k^t, \theta^t]$, which has a certain amount of personalization capability from λ, also obtains the generalization capability of (12). In other words, both personalization and generalization are achieved. Here, as λ grows, a larger number of global rounds is required to reduce the upper bound in (13); this is the cost for achieving a stronger personalization at the client-side. Note that the case with $\lambda = 1$ does not guarantee convergence, since the client-side models are not aggregated. On the other hand, the case with $\lambda = 0$ reduces to the bound of conventional FL.

B. Convergence Proof

Using $v_k^t = [\phi_k^t, h_k^t, \theta^t]$, we first define v^t as:

$$v^t = \frac{1}{K}\sum_{k=1}^{K} v_k^t. \tag{15}$$

By the L-smoothness of $F(v)$ and taking the expectation of both sides, we have

$$\mathbb{E}[F(v^{t+1})] - \mathbb{E}[F(v^t)] \leq \underbrace{\mathbb{E}[\langle\nabla F(v^t), v^{t+1} - v^t\rangle]}_{A} + \underbrace{\frac{L}{2}\mathbb{E}[\|v^{t+1} - v^t\|^2]}_{B}. \tag{16}$$

Step 1: Bounding A. We first rewrite A as follows:

$$A \underset{(a)}{=} -\eta_t\mathbb{E}\Big[\Big\langle\nabla F(v^t), \frac{1}{K}\sum_{k=1}^{K}\tilde{\nabla} F_k(v_k^t)\Big\rangle\Big] \tag{17}$$

$$\underset{(b)}{=} -\eta_t\mathbb{E}\Big[\Big\langle\nabla F(v^t), \frac{1}{K}\sum_{k=1}^{K}\nabla F_k(v_k^t)\Big\rangle\Big] \underset{(c)}{=} \underbrace{-\frac{\eta_t}{2}\mathbb{E}\Big[\|\nabla F(v^t)\|^2\Big]}_{A_1} - \frac{\eta_t}{2}\mathbb{E}\Big[\Big\|\frac{1}{K}\sum_{k=1}^{K}\nabla F_k(v_k^t)\Big\|^2 - \underbrace{\Big\|\nabla F(v^t) - \frac{1}{K}\sum_{k=1}^{K}\nabla F_k(v_k^t)\Big\|^2}_{A_2}\Big],$$

where (a) comes from $v^{t+1} - v^t = -\eta_t\frac{1}{K}\sum_{k=1}^{K}\tilde{\nabla} F_k(v_k^t)$, (b) follows from taking the expectation for the mini-batch, and (c) is obtained by utilizing $\|z_1 - z_2\|^2 = \|z_1\|^2 + \|z_2\|^2 - 2\langle z_1, z_2\rangle$.

We now focus on A_1. We can write

$$\begin{aligned}\|\nabla F(v^t)\|^2 &\underset{(d)}{\geq} \frac{1}{2}\|\nabla F(v_k^t)\|^2 - \|\nabla F(v_k^t) - \nabla F(v^t)\|^2 \\ &= \frac{1}{2}\|\nabla F(v_k^t)\|^2 - \Big\|\frac{1}{K}\sum_{i=1}^{K}(\nabla F_i(v_k^t) - \nabla F_i(v^t))\Big\|^2 \\ &\underset{(e)}{\geq} \frac{1}{2}\|\nabla F(v_k^t)\|^2 - L^2\|v_k^t - v^t\|^2\end{aligned} \tag{18}$$

for any k. Here, (d) comes from using $\|a + b\|^2 \leq 2\|a\|^2 + 2\|b\|^2$ and (e) comes from L-smoothness. Thus, we can bound A_1 as

$$\begin{aligned}A_1 &= -\frac{\eta_t}{2}\mathbb{E}\Big[\|\nabla F(v^t)\|^2\Big] = -\frac{\eta_t}{2K}\sum_{k=1}^{K}\mathbb{E}\Big[\|\nabla F(v^t)\|^2\Big] \\ &\leq -\frac{\eta_t}{4K}\sum_{k=1}^{K}\mathbb{E}\Big[\|\nabla F(v_k^t)\|^2\Big] + \frac{\eta_t L^2}{2K}\sum_{k=1}^{K}\mathbb{E}[\|v_k^t - v^t\|^2].\end{aligned} \tag{19}$$

Authorized licensed use limited to: Hongik Univ. Downloaded on May 02,2025 at 07:59:09 UTC from IEEE Xplore. Restrictions apply.

For A_2, we have

$$\frac{\eta_t}{2}\mathbb{E}[A_2] = \frac{\eta_t}{2}\mathbb{E}\Big[\Big\|\frac{1}{K}\sum_{k=1}^{K}(\nabla F_k(v^t) - \nabla F_k(v_k^t))\Big\|^2\Big]$$
$$\underset{(f)}{\leq} \frac{\eta_t}{2K}\sum_{k=1}^{K}\mathbb{E}[\|\nabla F_k(v^t) - \nabla F_k(v_k^t)\|^2]$$
$$\underset{(g)}{\leq} \frac{\eta_t L^2}{2K}\sum_{k=1}^{K}\mathbb{E}[\|v^t - v_k^t\|^2], \tag{20}$$

where (f) holds due to the convexity of $\|\cdot\|^2$ and (g) holds due to the L-smoothness assumption.

Step 2: Bounding B**.** Now we bound the term B. By utilizing $v^{t+1} - v^t = -\eta_t \frac{1}{K}\sum_{k=1}^{K}\tilde{\nabla} F_k(v_k^t)$, we can write $B \leq \eta_t^2 L(\mathbb{E}[\|\frac{1}{K}\sum_{k=1}^{K}\nabla F_k(v_k^t)\|^2] + \mathbb{E}[\|\frac{1}{K}\sum_{k=1}^{K}\nabla F_k(v_k^t) - \frac{1}{K}\sum_{k=1}^{K}\tilde{\nabla} F_k(v_k^t)\|^2])$, where

$$\mathbb{E}\Big[\Big\|\frac{1}{K}\sum_{k=1}^{K}\nabla F_k(v_k^t) - \frac{1}{K}\sum_{k=1}^{K}\tilde{\nabla} F_k(v_k^t)\Big\|^2\Big] \tag{21}$$
$$\leq \frac{1}{K}\sum_{k=1}^{K}\mathbb{E}[\|\nabla F_k(v_k^t) - \tilde{\nabla} F_k(v_k^t)\|^2]\Big) \underset{(h)}{\leq} \frac{1}{K}\sum_{k=1}^{K}\sigma_k^2$$

and (h) results from Assumption 3.

By inserting the bounds of A and B to (16), and by employing a learning rate that satisfies $\eta_t \leq \frac{1}{2L}$, we obtain

$$\frac{\eta_t}{4K}\sum_{k=1}^{K}\mathbb{E}\Big[\|\nabla F(v_k^t)\|^2\Big] \leq \mathbb{E}[F(v^t)] - \mathbb{E}[F(v^{t+1})]$$
$$+ \frac{\eta_t^2 L}{K}\sum_{k=1}^{K}\sigma_k^2 + \underbrace{\frac{\eta_t L^2}{K}\sum_{k=1}^{K}\mathbb{E}[\|v_k^t - v^t\|^2]}_{C}. \tag{22}$$

Step 3: Bounding C**.** To bound C, we define

$$\psi_k^t = [\phi_k^t, h_k^t] \tag{23}$$
$$\hat{\psi}_k^{t+1} = \psi_k^t - \eta_t \tilde{\nabla}_\psi F_k(v_k^t), \tag{24}$$

$\psi^t = \frac{1}{K}\sum_{k=1}^{K}\psi_k^t$ and $\hat{\psi}^t = \frac{1}{K}\sum_{k=1}^{K}\hat{\psi}_k^t$. Then, we have

$$\psi_k^{t+1} = \lambda\hat{\psi}_k^{t+1} + (1-\lambda)\hat{\psi}^{t+1} \tag{25}$$

and $\frac{1}{K}\sum_{k=1}^{K}\|v_k^t - v^t\|^2 = \frac{1}{K}\sum_{k=1}^{K}\|\psi_k^t - \psi^t\|^2$. We can write

$$\frac{1}{K}\sum_{k=1}^{K}\|\psi_k^t - \psi^t\|^2 = \frac{1}{K}\sum_{k=1}^{K}\|(\psi_k^t - \hat{\psi}^t) - (\psi^t - \hat{\psi}^t)\|^2$$
$$\underset{(i)}{\leq} \frac{1}{K}\sum_{k=1}^{K}\|\psi_k^t - \hat{\psi}^t\|^2 \underset{(j)}{=} \frac{1}{K}\sum_{k=1}^{K}\|\lambda(\hat{\psi}_k^t - \hat{\psi}^t)\|^2$$
$$= \frac{\lambda^2}{K}\sum_{k=1}^{K}\|(\hat{\psi}_k^t - \psi^{t-1}) - (\hat{\psi}^t - \psi^{t-1})\|^2$$
$$\underset{(k)}{\leq} \frac{\lambda^2}{K}\sum_{k=1}^{K}\|\hat{\psi}_k^t - \psi^{t-1}\|^2$$
$$\underset{(l)}{=} \frac{\lambda^2}{K}\sum_{k=1}^{K}\|-\eta_{t-1}\tilde{\nabla}_\psi F_k(v_k^{t-1}) + (\psi_k^{t-1} - \psi^{t-1})\|^2 \tag{26}$$

where (i) and (k) come from $\mathbb{E}[\|z - \mathbb{E}[z]\|^2] \leq \mathbb{E}[\|z\|^2]$, (j) follows from (25) and (l) results from $\hat{\psi}_k^{t+1} = \psi_k^t - \eta_t\tilde{\nabla}_\psi F_k(v_k^t)$. Now following the proof of Lemma 4 of [39] and utilizing Assumption 2, when $\eta_t = \frac{\eta_0}{a+t}$ and $a = \frac{c+4}{1-\lambda^2}$ for some constant $c > 0$, we have $\frac{1}{K}\sum_{k=1}^{K}\mathbb{E}[\|v_k^t - v^t\|^2] \leq \frac{16(c+4)G^2\lambda^2(2-\lambda^2)}{c(1-\lambda^2)^2}$.

Step 4: Telescoping sum. Finally, after inserting the result of Step 3 into (22), we have

$$\frac{\eta_t}{4K}\sum_{k=1}^{K}\mathbb{E}\Big[\|\nabla F(v_k^t)\|^2\Big] \leq \mathbb{E}[F(v^t)] - \mathbb{E}[F(v^{t+1})]$$
$$+ \frac{\eta_t^2 L}{K}\sum_{k=1}^{K}\sigma_k^2 + \frac{16\eta_t^3(c+4)G^2\lambda^2(2-\lambda^2)}{c(1-\lambda^2)^2}. \tag{27}$$

After summing up for $t = 0, 1 \ldots, T-1$ and dividing both sides by $\Gamma_T = \sum_{t=0}^{T-1}\eta_t$, with some manipulations, we obtain (13). This completes the proof of Theorem 1.

V. Inference-Time Analysis and Model Splitting

In this section, we analyze the storage, computation, communication and time required during the inference stage.

A. Notations and Assumptions

Let P_C and P_S be the available computing powers of each client and the server, respectively. Let $|\phi|$, $|\theta|$, $|h|$ be the numbers of parameters of ϕ, θ, h, respectively. Considering $P_C \ll P_S$ in practice, we split the model $w = [\phi, \theta]$ such that the size of client-side component ϕ is significantly smaller than the server-side component θ, i.e., $|\phi| \ll |\theta|$. Moreover, h is assumed to be a small classifier satisfying $|h| \ll |\phi|$ and $|h| \ll |\theta|$. To make our analysis tractable, we assume that the inference time (i.e., time required for forward propagation through the neural network) is proportional to the number of parameters of the model [28], [40]. For example, given a model with $|\phi| + |h|$ parameters and a test dataset of size $|D|$, the inference time at the client will be proportional to $\frac{(|\phi|+|h|)|D|}{P_C}$. One may also consider different latency models which is out of scope of this paper. We define R as the uplink data rate between a single client and the server. β is the portion of total test samples that are inferred at the client-side as a result of (11), while $1-\beta$ is the portion of test samples inferred at the server-side. Finally, q_c denotes the dimension of the cut-layer (i.e., output dimension of ϕ) and q denotes the size of the test sample (input dimension of the model). We assume the same q_c for all client layers for analytical tractability.

B. Resource and Latency Analysis

Table I compares our methodology with existing frameworks during inference. We present the derivations in the following:

Full model at the client. When the full model $w = [\phi, \theta]$ is implemented at individual clients, the required storage for each client is $|\phi| + |\theta|$. Hence, the client-side computational load becomes $(|\phi| + |\theta|)|D|$. Since all predictions are made at the client, no communication is required during inference. Hence, the inference time can be written as follows:

$$\tau_1 = \frac{(|\phi| + |\theta|)|D|}{P_C}. \tag{28}$$

Authorized licensed use limited to: Hongik Univ. Downloaded on May 02,2025 at 07:59:09 UTC from IEEE Xplore. Restrictions apply.

TABLE I: **Resources required and latency incurred at each client during inference:** comparing SplitGP with baselines.

Methods	Storage	Computation	Communication	Inference time
Full model at the server-side	0	0	$q\|D\|$	$\frac{q\|D\|}{R} + \frac{(\|\phi\|+\|\theta\|)\|D\|}{P_S}$
Full model at the client-side	$\|\phi\|+\|\theta\|$	$(\|\phi\|+\|\theta\|)\|D\|$	0	$\frac{(\|\phi\|+\|\theta\|)\|D\|}{P_C}$
Proposed framework (SplitGP)	$\|\phi\|+\|h\|$	$(\|\phi\|+\|h\|)\|D\|$	$\beta q_c\|D\|$	$\frac{(\|\phi\|+\|h\|)\|D\|}{P_C} + \frac{\beta q_c\|D\|}{R} + \frac{\beta\|\theta\|\|D\|}{P_S}$

Full model at the server. When the full model is deployed at the edge server, client-side storage is unused and the client-side computational load is also zero. Since the entire test set must be sent to the server, the required communication load during inference becomes $q|D|$. The inference time can be written as the sum of communication time and server-side computation time as follows:

$$\tau_2 = \frac{q|D|}{R} + \frac{(|\phi|+|\theta|)|D|}{P_S}. \tag{29}$$

Proposed SplitGP. In our approach, the required storage space at each client is $|\phi|+|h|$ while the server-side storage is $|\theta|$. The client-side computation is written as $(|\phi|+|h|)|D|$. Given the cut-layer dimension q_c, β portion of test samples are predicted at the server-side, requiring a communication load of $\beta q_c|D|$. For inference time, given a test sample z, forward propagation is first performed at the client-side, which has a latency of $\frac{|\phi|+|h|}{P_C}$. If $E_t(k) \leq E_{th}$ (with probability $1-\beta$), the prediction is made at the client which requires no additional time. Otherwise, with probability β, each client sends the output feature of ϕ_k to the server, which requires an additional latency of $\frac{q_c}{R}$ for communication and $\frac{|\theta|}{P_S}$ for server-side computation. Hence, the latency of SplitGP is written as

$$\tau = \frac{(|\phi|+|h|)|D|}{P_C} + \frac{\beta q_c|D|}{R} + \frac{\beta|\theta||D|}{P_S}. \tag{30}$$

Based on this analysis, we pose the following two questions: (i) How should we split the model w into ϕ and θ in practice? (ii) When is our framework with split models beneficial compared to other baselines in terms of inference time?

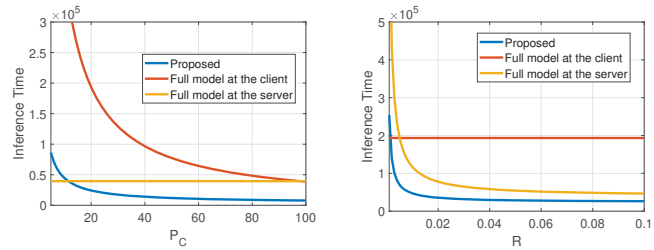
C. Model Splitting and Feasible Regimes

For the first question above, note that model splitting gives a trade-off between client-side personalization capability and inference time: as we increase the size of the client-side component ϕ, personalization improves but the inference time increases according to (30). Let $|\phi_{\text{min}}|$ be the minimum size of ϕ to achieve a desired level of personalization capability at the client, which must be selected empirically based on the local dataset and observed ML task difficulty. We also let τ' be the latency per test sample that the system should support. Based on these constraints, we state the following result:

Proposition 1. *From the latency constraint $\frac{\tau}{|D|} \leq \tau'$ and the personalization constraint $|\phi| \geq |\phi_{min}|$, the feasible model splitting regime for the client-side component is given as*

$$|\phi_{min}| \leq |\phi| \leq \frac{P_C P_S(\tau' - \frac{\beta q_c}{R} - \frac{|h|}{P_C} - \frac{\beta|w|}{P_S})}{P_S - \beta P_C}. \tag{31}$$

Note that $P_S - \beta P_C > 0$ holds since $P_C < P_S$ and $\beta \leq 1$. When improving accuracy prioritized over improving latency, we can split the model to satisfy $|\phi| = \frac{P_C P_S(\tau' - \frac{\beta q_c}{R} - \frac{|h|}{P_C} - \frac{\beta|w|}{P_S})}{P_S - \beta P_C}$, i.e., increase the size of the client-side component as much as possible. When latency is prioritized, we choose $|\phi| = \phi_{\text{min}}$, to minimize the inference time while achieving the minimum required personalization capability at the client-side.



(a) Inference time versus P_C (b) Inference time versus R

Fig. 3: Inference time depending on client-side computing power P_C and communication rate R. Our framework demonstrates significant advantage compared to the baselines in various power-rate regimes.

Now we turn to the second question, considering a fixed model splitting $w = [\phi, \theta]$. We first compare with the case where the full model is deployed at the client. From (28) and (30), we have the following proposition:

Proposition 2. *We have $\tau \leq \tau_1$ if and only if*

$$P_C \leq \frac{|\theta| - |h|}{\beta(\frac{q_c}{R} + \frac{|\theta|}{P_S})}. \tag{32}$$

The above result indicates that our solution is faster than the baseline when the client-side computing power P_C is smaller than a specific threshold. This makes intuitive sense because deploying the full model at the client-side incur significant inference latency when P_C is small (e.g., low-cost IoT devices).

Second, we compare with the baseline where the full model is implemented at the edge server during inference. Based on (29) and (30), we state the following proposition:

Proposition 3. *We have $\tau \leq \tau_2$ if and only if*

$$R \leq \frac{q - \beta q_c}{\frac{|\phi|+|h|}{P_C} - \frac{(1-\beta)(|\phi|+|\theta|)}{P_S}}. \tag{33}$$

According to (33), our solution is beneficial when the communication rate R is smaller than a specific value, since this baseline requires transmission of all test samples from client to server.

Fig. 3 shows the inference times of the models in Table I with $|\phi| = 387,840$, $|\theta| = 3,480,330$, $|h| = 23,050$, which corresponds to the convolutional neural network (CNN) that is utilized for experiments in the next section. Other parameters are $P_S = 100$, $P_C = 20$, $R = 1$, $\beta = 0.1$, $|D| = 1$. It can be seen that our framework achieves smaller inference time compared to existing baselines in various P_C and R regimes.

Authorized licensed use limited to: Hongik Univ. Downloaded on May 02,2025 at 07:59:09 UTC from IEEE Xplore. Restrictions apply.

VI. Experimental Results

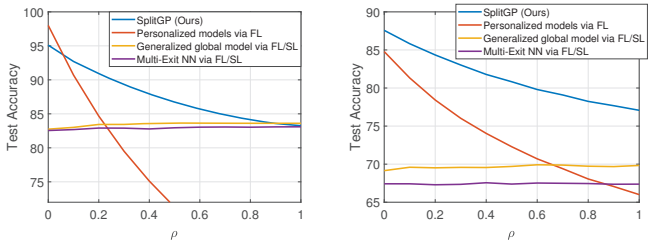
We evaluate our method on Fashion-MNIST (FMNIST) [41] and CIFAR-10 [42]. Both datasets contain 10 classes. We utilize a CNN with 5 convolutional layers and 3 fully connected layers for FMNIST dataset. For CIFAR-10, we adopt VGG-11.

Implementation. We consider $K = 50$ clients. To model non-IID data distributions, following the setup of [3], we first sort the overall train set based on classes and divide it into 100 shards. We then randomly allocate 2 shards to each client. We used a learning rate of $\eta = 0.01$ for all schemes. In each global round, each client updates its model for one epoch with a mini-batch size of 50, and cross-entropy loss is utilized throughout the training process. Moreover, we set $\lambda = 0.2$ and choose the optimal $E_{th} \in \{0.05, 0.1, 0.2, 0.4, 0.8, 1.2, 1.6, 2.3\}$ unless otherwise stated. We train the CNN model with FMNIST for 120 global rounds and VGG-11 model with CIFAR-10 for 800 global rounds. For our scheme, we split the full CNN model (for FMNIST) such that the client-side ϕ contains 4 convolutional layers ($|\phi| = 387,840$) and the server-side θ contains 1 convolutional layer and 3 fully connected layers ($|\theta| = 3,480,330$). The fully connected layer with size $|h| = 23,050$ is utilized as the auxiliary classifier. We also split the VGG-11 as $|\phi| = 972,554$ and $|\theta| = 8,258,560$, and adopt the fully connected layer with size $|h| = 10,250$ as a classifier.

Baselines. We compare SplitGP with the following baselines. First, we consider the personalized FL scheme proposed in [19], where the trained personalized models are deployed at individual clients during inference. We also consider a generalized global model constructed via conventional FL [3] as well as SplitFed [26]. Note that FL and SplitFed produce the same model while SplitFed can save storage and computation resources during training. This generalized global model can be deployed either at the client or at the server. Finally, we consider a multi-exit neural network that has two exits, one at the client-side and the other at the server-side, constructed via FL or SL. For a fair comparison, FedAvg [3] is adopted for the model aggregation process of all schemes.

Evaluation. When training is finished, the overall performance is measured by averaging the local test accuracies of all clients. We construct the local test set of each client as the union of the main test samples and the out-of-distribution test samples. The main test samples are constructed by selecting all test samples of the main classes, e.g., if client k has only classes 1 and 2 in its local data, all the test samples with classes 1 and 2 in the original test set are selected to construct the main test samples. When constructing the out-of-distribution test samples, we utilize the relative portion of out-of-distribution test samples ρ defined in (2). Given the main test samples, a fraction ρ of out-of-distribution samples are selected from the original test set. We reiterate that the previous works on personalized FL adopted $\rho = 0$ for evaluation.

Main result 1: Effect of out-of-distribution data. We first observe Fig. 4 and Table II, which show the performance of each scheme depending on the relative portion of out-of-distribution data ρ during inference. We have the following key observations. First, the performance of the generalized global model and the multi-exit neural network constructed via FL/SL do not dramatically change with varying ρ. This implies that all classes pose a similar level of difficulty for classification, which is consistent with the class-balanced nature of FMNIST and CIFAR-10. It can be also seen that the performance of personalized FL is significantly degraded as ρ grows, since personalized models are designed to improve the performance on the main classes, not the out-of-distribution classes. Finally, it is observed that SplitGP captures both personalization and generalization capabilities: due to the personalization capability, our scheme achieves a strong performance when ρ is small, and due to the generalization capability, our scheme is more robust against ρ compared to personalized FL.



(a) FMNIST (b) CIFAR-10

Fig. 4: Test accuracy vs. ρ. By capturing both personalization and generalization, SplitGP has advantages for most settings of ρ.

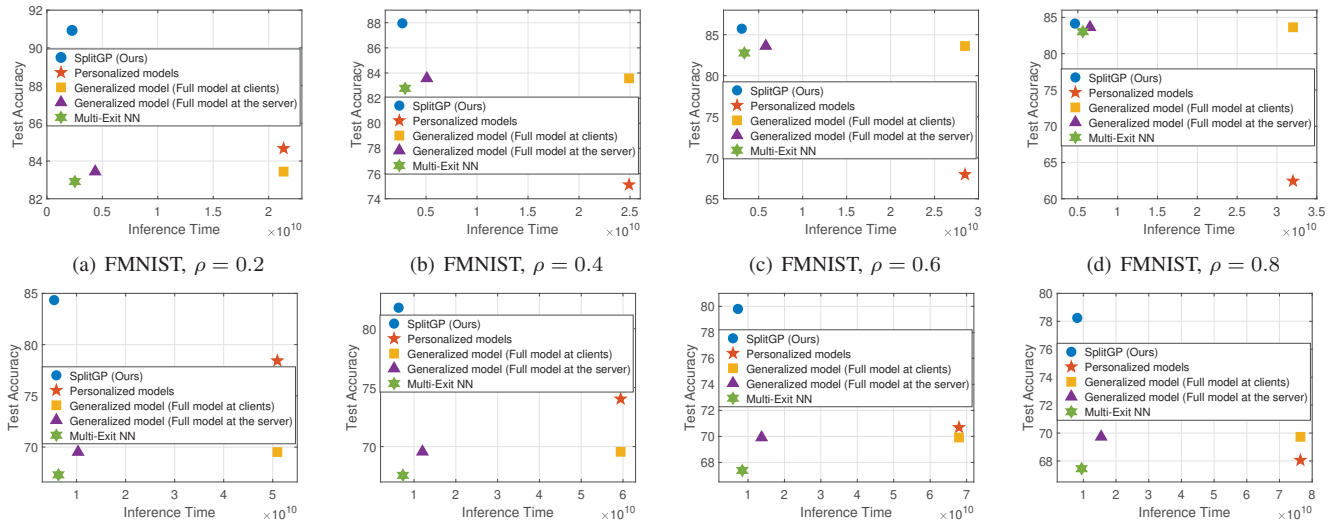
TABLE II: Effect of out-of-distribution test samples on FMNIST.

Methods	$\rho = 0$	$\rho = 0.2$	$\rho = 0.4$	$\rho = 0.6$	$\rho = 0.8$
Personalized FL	**98.00%**	84.67%	75.11%	67.96%	62.43%
Generalized FL	82.75%	83.44%	83.57%	83.62%	83.64%
SplitGP (Ours)	95.10%	**90.93%**	**87.95%**	**85.74%**	**84.15%**

Main result 2: Latency, accuracy, and resource improvements. Fig. 5 shows the achievable accuracy-latency performance of the different schemes. For personalized FL, the models are deployed at individual clients while the generalized global model can be deployed either at the client-side or at the server-side. To evaluate the inference time, we compute the latency from Table I by setting $P_C = 20$, $P_S = 100$, $R = 1$, as in Fig. 3. It can be seen that SplitGP achieves the best accuracy with smallest inference time for most values of ρ, confirming the advantage of our solution. Note that this performance advantage is achieved with considerable storage savings at the clients; compared to the case where the full model $w = [\phi, \theta]$ is deployed at each client, our scheme only requires 10.62% and 10.64% of the storage space for FMNIST and CIFAR-10, respectively, by saving only the client-side component ϕ. The communication load is also significantly reduced compared to others; for example, when $\rho = 0.8$ in FMNIST, our scheme achieves the best performance while inferring only 20.30% of the test samples at the server.

Ablation 1: Effect of λ and E_{th}. In Fig. 6(a) and Table III, we study the effect of λ which controls the weights for personalization and generalization. When λ is relatively large, the weight for the personalized client-side model increases, which leads to stronger personalization. However, the performance degrades as ρ increases, since the scheme with large λ

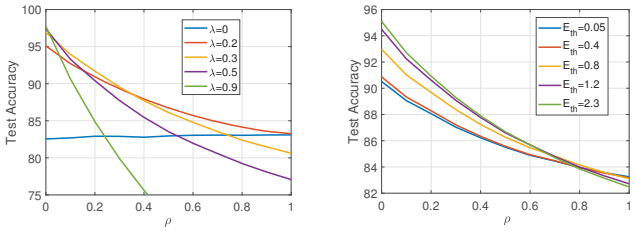
Authorized licensed use limited to: Hongik Univ. Downloaded on May 02,2025 at 07:59:09 UTC from IEEE Xplore. Restrictions apply.



(a) FMNIST, $\rho = 0.2$ (b) FMNIST, $\rho = 0.4$ (c) FMNIST, $\rho = 0.6$ (d) FMNIST, $\rho = 0.8$

(e) CIFAR-10, $\rho = 0.2$ (f) CIFAR-10, $\rho = 0.4$ (g) CIFAR-10, $\rho = 0.6$ (h) CIFAR-10, $\rho = 0.8$

Fig. 5: Achievable accuracy-latency tradeoff. Our scheme achieves the best accuracy with smallest inference time for most settings of ρ on both datasets, underscoring the ability of SplitGP to provide personalization and generalization while reducing inference resource requirements.



(a) Effect of λ. (b) Effect of E_{th}.

Fig. 6: Effects of λ and E_{th} in SplitGP for FMNIST. Larger λ leads to stronger personalization while smaller λ leads to stronger generalization. A larger E_{th} is a good option when ρ is small, while a smaller E_{th} achieves a better performance when ρ is large enough.

lacks generalization capability. In general, the best λ depends on the ρ value. Without prior information, i.e., assuming ρ is uniform in the range of $[0, 1]$, $\lambda = 0.2$ gives the best expected accuracy. On the other hand, if we have prior knowledge that ρ is uniform in $[0, 0.2]$, $\lambda = 0.3$ is a better option.

Now we observe the effect of E_{th} in Fig. 6(b). Similar to λ, one can choose an appropriate E_{th} given the expected ρ (or the range of ρ). When ρ is small, a large E_{th} performs well, which means that a relatively large number of samples should be predicted at the client-side to achieve the highest accuracy. On the other hand, when ρ is large, smaller E_{th} performs well which indicates that a large number of samples should be predicted at the server. These observations are consistent with our intuition that the main test samples should be predicted at the client-side (with strong personalization) while the out-of-distribution samples should be predicted at the server (with strong generalization), to achieve the most robust performance.

Ablation 2: Performance of each component. Finally, we consider the performance of different components of our model. Table IV compares the performance of the client-side model (ϕ_k combined with h_k) and the full model (ϕ_k combined with θ) with the complete SplitGP on FMNIST. Due to the personalization capability, it can be seen that SplitGP relies on the client model when ρ is small. As ρ increases, SplitGP relies on both the client model and the server model to achieve generalization and personalization jointly.

TABLE III: Effect of λ on FMNIST. The value of λ should be chosen to achieve both generalization and personalization, depending on the expected range of ρ.

Methods	$\rho = 0$	$\rho = 0.2$	$\rho = 0.4$	$\rho = 0.6$	$\rho = 0.8$
$\lambda = 0.2$	95.10%	90.93%	**87.95%**	**85.74%**	**84.15%**
$\lambda = 0.3$	96.93%	**91.69%**	87.70%	84.79%	82.39%
$\lambda = 0.5$	97.39%	90.40%	85.46%	81.96%	79.24%
$\lambda = 0.9$	**97.75%**	84.87%	75.62%	68.79%	63.46%

TABLE IV: Performance of the client-side model and the full model on FMNIST. Our scheme takes the benefits of both models.

Methods	$\rho = 0.2$	$\rho = 0.4$	$\rho = 0.6$	$\rho = 0.8$
Client model (SplitGP)	90.93%	87.90%	85.68%	83.85%
Full model (SplitGP)	88.06%	86.22%	84.89%	83.96%
Overall performance (SplitGP)	**90.93%**	**87.95%**	**85.74%**	**84.15%**

VII. Conclusion

In this paper, we proposed a hybrid federated and split learning methodology, termed SplitGP, which captures both personalization and generalization needs for reliable/efficient inference at resource-constrained clients. We analytically characterized the convergence of our algorithm, and provided guidelines on model splitting based on inference time analysis. Experimental results on real-world datasets confirmed the advantage of SplitGP in practical settings where each client needs to make predictions frequently for its main classes but also occasionally for its out-of-distribution classes.

Acknowledgement

This work was supported by IITP funds from MSIT of Korea (No. 2020-0-00626, No. 2021-0-02201), NRF (No. 2019R1I1A2A02061135, No. 2022R1A4A3033401), NSF CNS-2146171 and DARPA D22AP00168-00. Minseok Choi is the corresponding author.

Authorized licensed use limited to: Hongik Univ. Downloaded on May 02,2025 at 07:59:09 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] P. Kairouz, H. B. McMahan, B. Avent, A. Bellet, M. Bennis, A. N. Bhagoji, K. Bonawitz, Z. Charles, G. Cormode, R. Cummings *et al.*, "Advances and open problems in federated learning," *Foundations and Trends® in Machine Learning*, vol. 14, no. 1–2, pp. 1–210, 2021.

[2] T. Li, A. K. Sahu, A. Talwalkar, and V. Smith, "Federated learning: Challenges, methods, and future directions," *IEEE Signal Processing Magazine*, vol. 37, no. 3, pp. 50–60, 2020.

[3] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Artificial Intelligence and Statistics*, 2017, pp. 1273–1282.

[4] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, "Federated optimization in heterogeneous networks," *Proceedings of Machine Learning and Systems*, vol. 2, pp. 429–450, 2020.

[5] H. Wang, M. Yurochkin, Y. Sun, D. Papailiopoulos, and Y. Khazaeni, "Federated learning with matched averaging," in *International Conference on Learning Representations*, 2020.

[6] D. A. E. Acar, Y. Zhao, R. Matas, M. Mattina, P. Whatmough, and V. Saligrama, "Federated learning based on dynamic regularization," in *International Conference on Learning Representations*, 2020.

[7] S. P. Karimireddy, S. Kale, M. Mohri, S. Reddi, S. Stich, and A. T. Suresh, "Scaffold: Stochastic controlled averaging for federated learning," in *International Conference on Machine Learning*. PMLR, 2020, pp. 5132–5143.

[8] S. Wang, T. Tuor, T. Salonidis, K. K. Leung, C. Makaya, T. He, and K. Chan, "Adaptive federated learning in resource constrained edge computing systems," *IEEE Journal on Selected Areas in Communications*, vol. 37, no. 6, pp. 1205–1221, 2019.

[9] M. M. Amiri and D. Gündüz, "Federated learning over wireless fading channels," *IEEE Transactions on Wireless Communications*, vol. 19, no. 5, pp. 3546–3557, 2020.

[10] M. Chen, Z. Yang, W. Saad, C. Yin, H. V. Poor, and S. Cui, "A joint learning and communications framework for federated learning over wireless networks," *IEEE Transactions on Wireless Communications*, vol. 20, no. 1, pp. 269–283, 2020.

[11] Y. Park, D.-J. Han, D.-Y. Kim, J. Seo, and J. Moon, "Few-round learning for federated learning," *Advances in Neural Information Processing Systems*, vol. 34, pp. 28 612–28 622, 2021.

[12] J. Park, D.-J. Han, M. Choi, and J. Moon, "Sageflow: Robust federated learning against both stragglers and adversaries," *Advances in Neural Information Processing Systems*, vol. 34, pp. 840–851, 2021.

[13] D.-J. Han, M. Choi, J. Park, and J. Moon, "Fedmes: Speeding up federated learning with multiple edge servers," *IEEE Journal on Selected Areas in Communications*, vol. 39, no. 12, pp. 3870–3885, 2021.

[14] H. H. Yang, Z. Liu, T. Q. Quek, and H. V. Poor, "Scheduling policies for federated learning in wireless networks," *IEEE transactions on communications*, vol. 68, no. 1, pp. 317–333, 2019.

[15] Y. Tu, Y. Ruan, S. Wagle, C. G. Brinton, and C. Joe-Wong, "Network-aware optimization of distributed learning for fog computing," in *IEEE INFOCOM 2020-IEEE Conference on Computer Communications*. IEEE, 2020, pp. 2509–2518.

[16] S. Wang, M. Lee, S. Hosseinalipour, R. Morabito, M. Chiang, and C. G. Brinton, "Device sampling for heterogeneous federated learning: Theory, algorithms, and implementation," in *IEEE INFOCOM 2021-IEEE Conference on Computer Communications*. IEEE, 2021, pp. 1–10.

[17] H. Wang, Z. Kaplan, D. Niu, and B. Li, "Optimizing federated learning on non-iid data with reinforcement learning," in *IEEE INFOCOM 2020-IEEE Conference on Computer Communications*. IEEE, 2020, pp. 1698–1707.

[18] V. Smith, C.-K. Chiang, M. Sanjabi, and A. S. Talwalkar, "Federated multi-task learning," *Advances in neural information processing systems*, vol. 30, 2017.

[19] Y. Deng, M. M. Kamani, and M. Mahdavi, "Adaptive personalized federated learning," *arXiv preprint arXiv:2003.13461*, 2020.

[20] A. Fallah, A. Mokhtari, and A. Ozdaglar, "Personalized federated learning with theoretical guarantees: A model-agnostic meta-learning approach," *Advances in Neural Information Processing Systems*, vol. 33, pp. 3557–3568, 2020.

[21] M. Zhang, K. Sapra, S. Fidler, S. Yeung, and J. M. Alvarez, "Personalized federated learning with first order model optimization," in *International Conference on Learning Representations*, 2021.

[22] T. Li, S. Hu, A. Beirami, and V. Smith, "Ditto: Fair and robust federated learning through personalization," in *International Conference on Machine Learning*. PMLR, 2021, pp. 6357–6368.

[23] H.-Y. Chen and W.-L. Chao, "On bridging generic and personalized federated learning for image classification," in *International Conference on Learning Representations*, 2021.

[24] P. Vepakomma, O. Gupta, T. Swedish, and R. Raskar, "Split learning for health: Distributed deep learning without sharing raw patient data," *arXiv preprint arXiv:1812.00564*, 2018.

[25] O. Gupta and R. Raskar, "Distributed learning of deep neural network over multiple agents," *Journal of Network and Computer Applications*, vol. 116, pp. 1–8, 2018.

[26] C. Thapa, P. C. M. Arachchige, S. Camtepe, and L. Sun, "Splitfed: When federated learning meets split learning," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 36, no. 8, 2022, pp. 8485–8493.

[27] C. He, M. Annavaram, and S. Avestimehr, "Group knowledge transfer: Federated learning of large cnns at the edge," *Advances in Neural Information Processing Systems*, vol. 33, 2020.

[28] D.-J. Han, H. I. Bhatti, J. Lee, and J. Moon, "Accelerating federated learning with split learning on locally generated losses," in *ICML 2021 Workshop on Federated Learning for User Privacy and Data Confidentiality. ICML Board*, 2021.

[29] S. Oh, J. Park, P. Vepakomma, S. Baek, R. Raskar, M. Bennis, and S.-L. Kim, "Locfedmix-sl: Localize, federate, and mix for improved scalability, convergence, and latency in split learning," in *Proceedings of the ACM Web Conference 2022*, 2022, pp. 3347–3357.

[30] S. Teerapittayanon, B. McDanel, and H.-T. Kung, "Distributed deep neural networks over the cloud, the edge and end devices," in *2017 IEEE 37th international conference on distributed computing systems (ICDCS)*. IEEE, 2017, pp. 328–339.

[31] ——, "Branchynet: Fast inference via early exiting from deep neural networks," in *2016 23rd International Conference on Pattern Recognition (ICPR)*. IEEE, 2016, pp. 2464–2469.

[32] H. Hu, D. Dey, M. Hebert, and J. A. Bagnell, "Learning anytime predictions in neural networks via adaptive loss balancing," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 33, no. 01, 2019, pp. 3812–3821.

[33] G. Huang, D. Chen, T. Li, F. Wu, L. van der Maaten, and K. Weinberger, "Multi-scale dense networks for resource efficient image classification," in *International Conference on Learning Representations*, 2018.

[34] H. Li, H. Zhang, X. Qi, R. Yang, and G. Huang, "Improved techniques for training adaptive deep networks," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2019, pp. 1891–1900.

[35] M. Phuong and C. H. Lampert, "Distillation-based training for multi-exit architectures," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2019, pp. 1355–1364.

[36] M. Lecuyer, V. Atlidakis, R. Geambasu, D. Hsu, and S. Jana, "Certified robustness to adversarial examples with differential privacy," in *2019 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2019, pp. 656–672.

[37] X. Li, K. Huang, W. Yang, S. Wang, and Z. Zhang, "On the convergence of fedavg on non-iid data," in *International Conference on Learning Representations*, 2020.

[38] A. Reisizadeh, A. Mokhtari, H. Hassani, A. Jadbabaie, and R. Pedarsani, "Fedpaq: A communication-efficient federated learning method with periodic averaging and quantization," in *International Conference on Artificial Intelligence and Statistics*. PMLR, 2020, pp. 2021–2031.

[39] D. Basu, D. Data, C. Karakus, and S. Diggavi, "Qsparse-local-sgd: Distributed sgd with quantization, sparsification and local computations," *Advances in Neural Information Processing Systems*, vol. 32, 2019.

[40] A. Canziani, A. Paszke, and E. Culurciello, "An analysis of deep neural network models for practical applications," *arXiv preprint arXiv:1605.07678*, 2016.

[41] H. Xiao, K. Rasul, and R. Vollgraf, "Fashion-mnist: a novel image dataset for benchmarking machine learning algorithms," *arXiv preprint arXiv:1708.07747*, 2017.

[42] A. Krizhevsky, G. Hinton *et al.*, "Learning multiple layers of features from tiny images," 2009.

Authorized licensed use limited to: Hongik Univ. Downloaded on May 02,2025 at 07:59:09 UTC from IEEE Xplore. Restrictions apply.